

Costruire Agentic AutoML

Un progetto didattico per costruire passo dopo passo un sistema AutoML agentic realmente utile

Relazione del progetto - Edizione iniziale di progetto

OSSERVA

DECIDI

AGISCI

VALUTA

RICORDA

RESTITUISCI

Specifica iniziale del progetto - Agosto 2026

1. Sommario

Agentic AutoML è un progetto didattico versionato il cui obiettivo è costruire, in modo progressivo e trasparente, un agente AutoML per il machine learning supervisionato tabellare. Dati un dataset e una colonna target, il sistema maturo dovrà riconoscere se il problema è classificazione binaria, classificazione multiclasse o regressione; analizzare i dati; eseguire esperimenti controllati; confrontare strategie di preprocessing e modelli; ottimizzare le soluzioni promettenti; e restituire la migliore pipeline valida insieme alla cronologia completa delle decisioni.

Il progetto è intenzionalmente semplice all'inizio. Ogni versione introduce una sola capacità principale e mantiene il più possibile stabili i componenti già compresi.

2. Obiettivo e perimetro del progetto

- **Didattico:** rendere l'architettura dell'agente osservabile e comprensibile direttamente dal notebook.
- **Professionale:** usare validazione corretta, riproducibilità, tracciamento degli esperimenti e limiti espliciti.
- **Utile:** arrivare in un numero contenuto di versioni a un sistema capace di produrre una buona soluzione ML tabellare su dataset in stile Kaggle.
- **Tracciabile:** non restituire soltanto uno score; conservare cosa è stato provato, cosa è fallito, cosa ha migliorato il risultato e perché è stato scelto il metodo finale.

Confini attuali

- Le prime versioni non hanno l'obiettivo di superare librerie AutoML mature.
- Il progetto inizialmente riguarda solo machine learning supervisionato tabellare.
- LLM e framework agentici sono esclusi volutamente dalle prime versioni per rendere visibile il ciclo decisionale.

3. Metodo didattico

Il progetto riprende il metodo progressivo utilizzato nel precedente Building AI Agent: **una versione, un concetto, un miglioramento architetturale**. Ogni versione deve già funzionare end-to-end e una nuova astrazione viene introdotta solo quando emerge un limite concreto. La complessità viene aggiunta per risolvere un bisogno della versione precedente, non per anticipare l'architettura finale.

Principi di progettazione

- Ogni notebook funziona autonomamente dall'alto verso il basso.
- Le funzioni restano corte ed esplicite; le astrazioni arrivano solo quando risolvono un problema visibile.
- La funzione principale dell'agente coordina i componenti e resta leggibile quasi come pseudocodice.
- Gli output mostrano stato e cronologia degli esperimenti, non soltanto riepiloghi raffinati.
- Ogni nuova capacità viene dimostrata su un dataset o un esperimento scelto per renderne evidente il valore.

4. Architettura dell'agente

L'architettura matura ruota attorno a un piccolo ciclo ricorrente. Le prime versioni ne implementano solo una parte; le successive arricchiscono progressivamente ogni fase.

1. Observe	Analizza struttura del dataset, comportamento del target ed evidenze già prodotte.
2. Decide	Sceglie il prossimo esperimento utile: metrica, validazione, preprocessing, modello o tuning.
3. Act	Esegue un esperimento limitato tramite funzioni piccole ed esplicite.
4. Evaluate	Misura le prestazioni in modo coerente e rileva esperimenti non validi o falliti.
5. Remember	Salva configurazione, score, durata, stato e motivazione della decisione.
6. Return	Seleziona la migliore pipeline valida e spiega il percorso che ha portato alla scelta.

5. Roadmap delle versioni

La roadmap è volutamente compatta: otto versioni sono sufficienti per passare da una baseline minima a un sistema realmente utile senza creare decine di traguardi artificiali.

Versione	Nome	Cosa cambia	Lezione principale
V01	Baseline Agent	Riconosce il task supervisionato e allena una baseline LightGBM veloce.	Agente end-to-end minimo
V02	Data Inspector	Analizza target, tipi, missing, cardinalità e dimensione del dataset.	Osservare prima di agire
V03	Preprocessing Agent	Prova poche strategie di preprocessing e conserva gli esiti.	Decisioni sui dati
V04	Model Selection	Confronta diverse famiglie di modelli con valutazione coerente.	Scelta del modello su evidenza
V05	Smart Validation	Sceglie metriche e validazione più robuste; introduce il leakage.	Valutazione affidabile
V06	Feature Engineering	Testa trasformazioni semplici e motivate senza nascondere la logica.	Ricerca controllata di feature
V07	Optimization	Usa Optuna solo sui candidati promettenti con budget esplicito.	Tuning efficiente
V08	Senior Agent	Coordina esperimenti, interrompe strade improduttive e restituisce la pipeline migliore.	Sistema AutoML agenticamente utile

Regola: una nuova versione modifica la parte minima possibile della precedente.

6. Strategia dei dataset

Il progetto usa un unico Kaggle Dataset contenente piu file CSV. La difficolta cresce gradualmente e lo stesso dataset puo essere riutilizzato nelle versioni successive per mostrare se una nuova capacita produce un miglioramento misurabile. L agente riceve dataset e target; il tipo di task deve essere inferito dall agente e non letto da una risposta nascosta.

File	Task	Livello	Perche e incluso
01_simple_binary.csv	Classificazione binaria	Molto facile	Prima baseline pulita; feature numeriche e categoriche.
02_simple_regression.csv	Regressione	Molto facile	Replica la V01 con target continuo.
03_adult_income.csv	Classificazione binaria	Media	Missing, tipi misti e categoriche realistiche.
04_abalone.csv	Regressione	Media	Benchmark compatto con input numerici e categorici.
05_bank_marketing.csv	Classificazione binaria	Media	Dataset piu ampio, tipi misti e imbalance realistico.
06_wine_quality.csv	Regressione / ambiguo	Media	Target numerico utile per ragionare sulla definizione del task.
07_student_dropout.csv	Classificazione multiclasse	Difficile	Tre classi, imbalance e spazio di feature piu ricco.
08_bike_sharing.csv	Regressione	Difficile	Struttura temporale e candidati leakage intenzionali per le versioni avanzate.

I primi due file sono dataset sintetici del progetto. La raccolta reale e pianificata attorno a benchmark tabellari pubblici e noti, confezionati come CSV normalizzati in un unico Kaggle Dataset.

7. Stile di notebook, codice e output

- **Struttura notebook:** Goal, Version N, Imports, Data, Components, Agent, Run, Results, Notes.
- **Markdown:** spiegazioni brevi su cosa cambia, perche cambia e cosa resta invariato.
- **Codice:** funzioni piccole, nomi espliciti, dipendenze minime e nessuna gerarchia di classi prematura.
- **State:** semplici dizionari Python all inizio, da arricchire solo quando serve una storia degli esperimenti piu completa.
- **Risultati:** traccia decisionale grezza piu tabelle o grafici concisi quando aggiungono informazione reale.

Goal	Version N	Imports	Data	Components	Agent	Run	Results	Notes
------	-----------	---------	------	------------	-------	-----	---------	-------

8. Stato degli esperimenti e traccia decisionale

Un requisito centrale e l osservabilita. Ogni esperimento dovra conservare almeno: identificatore, task e metrica, strategia sui dati, modello, iperparametri, schema di validazione, score, durata, stato, motivo dell eventuale fallimento e decisione successiva. Questa storia permettera all agente finale di spiegare non solo *cosa ha vinto*, ma *come ci e arrivato*.

experiment_id | task | metrica | preprocessing | modello | parametri | validazione | score | durata | stato | decisione

9. Capacita finale attesa

- Riconoscere regressione, classificazione binaria e classificazione multiclasse.
- Analizzare feature numeriche e categoriche, missing, cardinalita, target e dimensione del dataset.
- Scegliere metriche e strategie di validazione adeguate.
- Confrontare diversi preprocessing e famiglie di modelli.
- Individuare leakage evidente e setup sperimentali non validi nelle versioni mature.
- Usare Optuna solo dopo aver ristretto la ricerca ai candidati promettenti.
- Restituire la migliore pipeline riproducibile e una cronologia leggibile degli esperimenti.

10. Stato attuale

Questo documento descrive il progetto iniziale prima dell'implementazione della V01. La raccolta benchmark e lo schema dei nomi sono stati definiti e il prossimo traguardo di sviluppo e **V01 - Baseline Agent**. I risultati sperimentali sono volutamente assenti da questa edizione e verranno aggiunti soltanto dopo essere stati prodotti dai notebook eseguiti.

Prossimo traguardo

Costruire V01 come sistema completo minimo: caricare un CSV, ricevere il target, inferire in modo semplice il tipo di task, allenare una baseline LightGBM veloce, valutarla e restituire uno state trasparente.

Questa relazione è un documento vivo del progetto. Deve evolvere solo quando le versioni eseguite dei notebook producono nuove evidenze, decisioni e risultati.

Infografica del progetto

